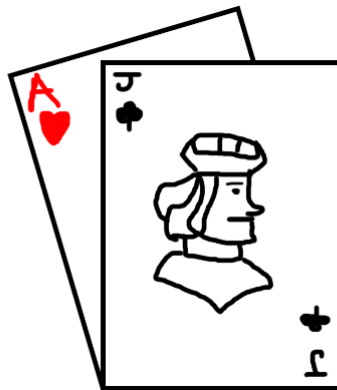


# Introduction to computer networking

First part of the assignment

Academic year 2026-2027



## Abstract

In this assignment, students will have to implement a client-server application using Java Sockets. The server is a game platform, i.e. it allows a user to play a simplified game of Blackjack (21) against the house. The client will interact with the user to display the game status, place bets, and decide whether to hit or stand.

Sections 1 and 2 define your assignment objectives. Sections 3 and 4 summarize the technologies we use. Section 5 explains what testing evidence your report must include. Section 6 describes the report. Section 7 presents the practical guidelines.

Students will work alone using the Java language. The **Hard Deadline** is October 25<sup>th</sup>, 2026.

## 1 Blackjack platform

As stated in the abstract, you will implement a Blackjack platform using Java Sockets. The server waits for TCP connections on a given port, receives requests through that connection when established, potentially updates its internal state accordingly, and sends a formatted response following the custom-made Blackjack protocol.

The server will remain available to accept connections and be able to handle concurrency. Each client plays its own game against the dealer, i.e. the house, which is played by the server itself. Each session corresponds to one TCP connection.

The client will interact with the user using terminal I/Os, send requests to the server, read its responses, and update its display accordingly.

Figure 1 shows an example of a potential output from the client, interacting with the user and with the server. You are free to design your own user interface as long as the message exchange with the **server respects the Blackjack Protocol** (cfr. 2).

## 1.1 Rules

This section describes the simplified rules of the Blackjack game that you will implement.

### 1.1.1 Overview

As soon as a client connects, the server shuffles a single standard 52-card deck and sets the player's bankroll to 100 chips. This deck is **never reshuffled** during the session: every round dealt during a session draws from what remains of that same deck.

Each round of play proceeds as follows: the player places a bet, the server deals two cards to the player and two cards to the dealer (one of the dealer's cards is dealt face-down and stays hidden until the dealer's turn), and the player then decides, one card at a time, whether to **hit** (draw an additional card) or **stand** (end their turn). Once the player stands (or busts), the dealer reveals its hidden card and plays according to a fixed rule (see 1.1.4). The hand closer to 21 without exceeding it wins the round.

Once a round has started, the user can decide to hit, stand, or quit the game. Between rounds, the user places a new bet to start the next round, as long as the remaining deck and bankroll allow it. The client software can help the user build their request and send it to the server. Upon receiving a request, the server updates the game status accordingly and sends back an appropriate response to the client.

The game session ends when the user quits, when the user runs out of chips, or when the deck no longer contains enough cards to deal a new round (see 1.1.6).

### 1.1.2 Card and hand values

Cards are represented in requests and responses as a rank immediately followed by a suit letter, with no space in between. Every card code is exactly **two characters** long:

- Rank: 2–9, T (Ten), J, Q, K, A.
- Suit: S (Spades), H (Hearts), D (Diamonds), C (Clubs).

For example, **AS** is the Ace of Spades and **TH** is the Ten of Hearts. A hidden (face-down) card is represented as **??**, also two characters.

Number cards are worth their face value, T/J/Q/K are worth 10, and an Ace is worth 11 unless that would make the hand exceed 21, in which case it is worth 1. A hand's total is always reported as this **best achievable value**. A **natural Blackjack** is an initial two-card total of 21 obtained with exactly two cards (an Ace and a ten-value card: T, J, Q, or K).

### 1.1.3 Player actions

The player can perform the following actions:

- **Bet**, to wager a chosen amount and start a new round.
- **Hit**, to draw an additional card (only while a round is in progress).
- **Stand**, to end their turn and let the dealer play (only while a round is in progress).
- **Bank**, to check the player's current bankroll, at any time.
- **Quit**, to end the session. The connection and resources must be properly released by the server.

**Hit** and **stand** are only available once a round has started, while **bet** is only available between rounds: there is no splitting of pairs and no doubling down. If the player's total exceeds 21 after a hit, the player busts and immediately loses the round. If a hit brings the player's total to exactly 21, the player's turn ends automatically and the dealer plays according to Section 1.1.4.

### 1.1.4 Dealer play

Once the player stands (without busting), the dealer reveals its hidden card and draws additional cards on its own, following a fixed strategy that requires no decision from the client: the dealer **hits while its total is 16 or less**, and **stands as soon as its total is 17 or more**. If the dealer busts (exceeds 21), the player wins the round.

If the player busts instead, the dealer does not draw any further card: its hidden card is simply revealed as-is and the round ends in a loss for the player.

### 1.1.5 Payouts

Chips are wagered at the start of each round and settled once the round is resolved:

- **Natural Blackjack** pays **3:2, rounded down to the nearest chip** (e.g. a 5-chip bet wins 7 chips, not 7.5). This payout only applies to a two-card 21 obtained directly on the deal without any hits.
- **Regular win** pays **1:1**. This payout applies when the player's total is higher than the dealer's without busting, or the dealer busts.
- **Push**: the bet is returned, no chips won or lost. The player and dealer have the same total (including a natural Blackjack for both).

- **Loss:** the bet is forfeited. The player busts, or the dealer's total is higher, or the dealer has a natural Blackjack and the player does not.

### 1.1.6 End of the game

Dealing a round requires four cards (two for the player, two for the dealer) and a positive bankroll. As soon as a round settles, the server checks whether both conditions still hold for a next round. If either fails – the remaining deck holds fewer than four cards, or the bankroll has dropped to zero (for instance after an all-in bet is lost) – the response that just settled the round also reports the end of the session, and the server closes the connection immediately afterward: no reshuffle ever occurs, and a bet cannot be placed once the bankroll reaches zero (see 2).

Card consumption during a round (extra hits by the player or the dealer) can, in rare cases, exhaust the deck *during* a round rather than between rounds – a hit or dealer draw takes the very last remaining card. Should the player's hit be the one to do so without busting, the round is settled immediately, as if a stand had followed – the same way a bust is settled as soon as it occurs, without waiting for an actual stand. If the dealer's own play would also require a card the deck no longer has, the dealer simply stops drawing and the round settles with the hands as they stand, after which the check above applies as usual.

### 1.1.7 Example session

Figure 1 shows an example of a game session. Notice that the dealer's hidden card is only revealed once the player stands, busts, or a natural Blackjack is resolved. You are free to design your own client user interface as long as the messages exchanged with the server respect the Blackjack Protocol (cfr. 2).

```

Connected. You start with 100 chips.
Commands:
bet <amount>  place a bet and start a new round
hit           draw an additional card
stand        end your turn and let the dealer play
bank         check your current bankroll
quit         leave the table

> bet 50
PLAYER JH KH 20
DEALER AH ??

> stand
PLAYER JH KH 20
DEALER AH 4S QD JS 25
WIN 50

> bet 150
PLAYER 8D 5H 13
DEALER 6C ??

> stand
PLAYER 8D 5H 13
DEALER 6C 9H 7S 22
WIN 150

> bank
BANKROLL 300

> quit
Leaving the table.
Goodbye.

```

Figure 1: Example of a potential console output.

## 1.2 Launch

The software is invoked on the command line with no additional arguments:

`java BlackjackServer` for the server and `java BlackjackClient` for the client.

The server listens on port  $2xxx$  – where  $xxx$  are the last three digits of your ULiege student ID.

### 1.3 Incorrect or incomplete commands

When dealing with network connections, you can never assume that the other side will behave as you expect.

It is thus your responsibility to check the validity of the client's request (and respond with the appropriate error code if something goes wrong) and to ensure that a malicious client won't make your server freeze by initiating a TCP connection and keeping it open for an indefinite period of time (see Section 3 for a counter-measure).

## 2 The Blackjack Protocol

The Blackjack Protocol (BP) is a custom-made protocol designed for this assignment only. This protocol is *text-oriented*, meaning that all exchanges are intended to be human-readable (as opposed to *binary*-encoded protocols). This protocol is a multiline protocol. Each request/response consists of a few text lines with **uppercase** command keywords, ending with `CRLF` (carriage return + line feed: `"\r\n"`) and ends in an empty line (i.e. each request ends with `CRLFCRLF`).

Here is a textual example of a short exchange, where **C:** denotes a client request and **S:** the corresponding server response. Note the doubled `CRLF` at the end of every message: the first closes the last content line, the second is the trailing empty line that marks the message boundary.

```
C: BET 10CRLFCRLF
S: PLAYER AS KH 21CRLF
  DEALER 9C 7D 16CRLF
  BLACKJACK 15CRLFCRLF

C: BANKCRLFCRLF
S: BANKROLL 115CRLFCRLF

C: BET 10CRLFCRLF
S: PLAYER 8D 6S 14CRLF
  DEALER 5H ??CRLFCRLF

C: HITCRLFCRLF
S: PLAYER 8D 6S 4C 18CRLF
  DEALER 5H ??CRLFCRLF

C: STANDCRLFCRLF
```

```
S: PLAYER 8D 6S 4C 18CRLF
    DEALER 5H 9D QC 24CRLF
    WIN 10CRLF
```

```
C: BET 125CRLF
```

```
S: PLAYER 9S 6C 15CRLF
    DEALER JD ??CRLF
```

```
C: STANDCRLF
```

```
S: PLAYER 9S 6C 15CRLF
    DEALER JD 8H 18CRLF
    LOSE 125CRLF
    GAME OVERCRLF
    BANKROLL 0CRLF
```

## 2.1 Client requests

The client will either place a bet, hit, stand, check its bankroll, or quit. It will thus send either:

- "BET *amount*" where *amount* is the amount of chips to bet and deal a new round. *amount* must be a NUMBER: a strictly positive base-10 integer without leading zeros.
- "HIT", to draw an additional card during the current round.
- "STAND", to end the player's turn and let the dealer play out the round.
- "BANK", to query the player's current bankroll.
- "QUIT", to request that the server close the connection.

Note that all these requests end with <sub>CRLF</sub>.

## 2.2 Server responses

Most responses contain a *Hand* pair with the following format:

- "PLAYER *card*<sub>1</sub> *card*<sub>2</sub> ... *total*", listing the player's cards followed by the current best total.
- "DEALER *card*<sub>1</sub> *card*<sub>2</sub> ... *total*", listing the dealer's cards followed by its current total. While the dealer's second card is still hidden, it is shown as "??" and *total* is also replaced by "??".
- Each line ends in <sub>CRLF</sub>.

A *RoundStatus* line is appended at the end of a *Hand* response whenever that response resolves the round: "BLACKJACK *amount*" (natural Blackjack, pays 3:2),

"WIN *amount*" (pays 1:1), "LOSE *amount*", or "PUSH 0" (bet returned, no chips won or lost). *amount* is the number of chips **won** for "BLACKJACK"/"WIN", or **forfeited** for "LOSE". No *RoundStatus* line is appended while a round is still in progress (i.e. the player may still hit or stand).

If settling a round brings the bankroll down to zero, or leaves the remaining deck with fewer than four cards, the server appends a *GameOver* block after the *RoundStatus* and closes the connection immediately afterward. *GameOver* is "GAME OVER" followed by the final "BANKROLL *amount*".

When receiving a "BET *amount*" request, the response depends on the game state:

- If *amount* is not a NUMBER, the server responds with "INVALID BET".
- If *amount* is a NUMBER and exceeds the current bankroll, the server responds with "INSUFFICIENT FUNDS".
- If a round is already in progress, the server responds with "ROUND IN PROGRESS". This rule has less priority than the two above: if the bet is invalid or exceeds the bankroll, the server responds with the corresponding error message even if a round is in progress.
- Otherwise, the server deducts the bet from the bankroll, deals two cards to the player and two to the dealer (dealer's second card hidden), and responds with *Hand*. *RoundStatus* and *GameOver* could also occur here: *RoundStatus* is appended if the deal itself produces a natural Blackjack, settling the round immediately, and *GameOver* is further appended if that settlement leaves the bankroll at zero or the remaining deck with fewer than four cards.

When receiving "HIT":

- If no round is in progress, the server responds with "NO ACTIVE ROUND".
- Otherwise, the server draws one card into the player's hand.
  - If the new total is less than 21, it responds with *Hand*.
  - If the new total is 21, the server moves to the next phase making the dealer play and it responds with *Hand* + *RoundStatus* and optional *GameOver*.
  - If the new total is more than 21, or that card was the **last one in the deck**, the server reveals the dealer's hidden card and responds with *Hand* + *RoundStatus* and optional *GameOver*.

When receiving "STAND":

- If no round is in progress, the server responds with "NO ACTIVE ROUND".
- Otherwise, the server reveals the dealer's hidden card, plays the dealer's hand according to 1.1.4, settles the bet, and responds with *Hand* + *RoundStatus* and optional *GameOver*.

When receiving a "BANK" request, the server responds with "BANKROLL *amount*" alone.

When receiving "QUIT", the server will close the connection to the client.

Finally, for any other request that it might receive, the server will answer with "WRONG" and keep the connection open.

### 3 Multi-Threading

A very simple solution for the server's code would be to accept a connection, handle the requests, then go back to accepting new connections, and so forth.

Now, imagine that a client behaves badly and sends an incomplete request. The server would then be blocked and would not be able to handle new connections.

A simple solution to handle this problem is to separate the code logic into separate threads. The main thread will loop on the `accept()` method and, when it succeeds, forward the task to a new Thread. That way, even if one client is erroneous, it will only impact one of the server's threads and other clients will be served correctly. In addition, using the method `setSoTimeout` on the socket will ensure that a client that stays silent for too long will trigger an exception on the server side.

We will see in the second part of the assignment that this method also has a flaw, but it will be considered acceptable for this first part.

### 4 Stream-oriented protocols (TCP)

The BP protocol relies on the TCP protocol, which is stream-oriented.

This has at least three implications:

1. TCP presents the application with a reliable, in-order byte stream: you can be sure that the data you read has not been lost, duplicated, or reordered, even though the underlying IP packets could have been.
2. The sender process could be *buffering* the output, i.e. the data that is requested to be sent might not have been effectively sent through the socket yet. This can be solved by calling the `flush()` method on the `OutputStream`.
3. **The data requested to be sent with one call to a write method will not necessarily be received with only one read method from the other side.** Several reads could be necessary to recover the entire information. This is because we are reading from a stream, not messages (as we would with UDP, for instance).

We thus have to find a way to delimit message boundaries on the stream. BP achieves this by having a blank line (i.e. two consecutive CRLF) after each full request or response – a single CRLF only terminates one line within a message, since a response can span several lines. In practice, you should thus use a class that takes multiple reads into account (such as `BufferedReader`) or perform multiple reads if using a low-level `read`.

## 5 Testing

You are expected to test your implementation: the program works as expected, respects the game rules, handles edge cases, malicious behavior, and the exchanged messages match the Blackjack Protocol.

Your report must show evidence of this testing, not merely describe the code. In particular:

- Describe the tools you used: a debugger, a network inspection/sniffing tool (e.g. Wireshark, tcpdump, or a raw `netcat/telnet` session), automated test cases, or any custom test client or script you wrote for this purpose.
- For each scenario you tested, briefly describe what you did and what you observed so the reader can see that the behavior matches the protocol.
- Explain how a reader could reproduce your tests: which commands to run, which inputs to provide, and what output to expect. This does not need to be exhaustive, but should be precise enough that the tests can actually be repeated.

## 6 Report

Your program will be completed with a PDF report that follows the **provided template** and doesn't exceed **4 pages**. The report will address the following points:

**Software architecture:** How have you broken down the problem to come to the solution? Name the major classes and their roles.

**TCP stream:** How did you **read** (not write) the requests from the TCP stream? You are allowed to provide the source code that handles this operation.

**Multi-thread organisation:** How did you achieve multi-threading in your program? You may also include source code to justify your answer. Did you synchronize the activity of the different threads?

**Robustness:** How did you make your server robust? How did you manage the different types of exceptions that your server might receive? How did you deal with the misbehavior of a client? How did you ensure that your server will not crash or freeze when a client misbehaves?

**Tests:** How did you test your program? What tools did you use? How can a reader reproduce your tests? You are allowed to provide excerpts of the exchanged messages or screenshots to illustrate your answer.

## 7 Guidelines

- You will implement the programs using Java 17, with packages `java.lang`, `java.io`, `java.net` and `java.util`.
- You will ensure that your program can be terminated at any time simply using CTRL+C, and avoid the use of ShutdownHooks
- You will not manipulate any file on the local file system.
- You will ensure your main class is named `BlackjackServer` (resp. `BlackjackClient`), located in `BlackjackServer.java` (resp. `BlackjackClient.java`) at the **root of the archive**, and does not contain any **package** instruction. You are allowed to create more classes and `.java` files than these two imposed ones.
- You will not cluster your program in packages or directories. All `.java` files should be found in the same directory.

**Submissions that do not observe these guidelines could be ignored during evaluation.**

Your commented source code alongside the report will be delivered no later than 25th October 2026 (**Hard deadline**) to the Montefiore Submission platform (<https://submit.montefiore.uliege.be/>) as a `.zip` package.

An oral examination will be organised at the end of the semester for the second part of the project. During this oral, you will be asked some questions to verify that you understand the code you submitted. Insufficient understanding of your own submission may result in a reduction of your grade.

## 8 Plagiarism

Plagiarism is the practice of taking someone else's work or ideas and passing them off as one's own. According to ULiege regulations, plagiarism is a fraud and is sanctioned with a grade of 0/20 for the course. In particular, giving his/her source code to another student, or copying the source code from another student's submission, past or present, is strictly prohibited. Copying from an external source is allowed if the part that is copied is well identified and the source cited. Note that, in that case, the cited material will not award any points in the grading. In case of suspicion of fraud, an oral examination will be conducted to determine if the project is indeed the student's work.

Good programming...